



# 算法分析与设计

朱容波

Email: [rongbozhu@126.com](mailto:rongbozhu@126.com)

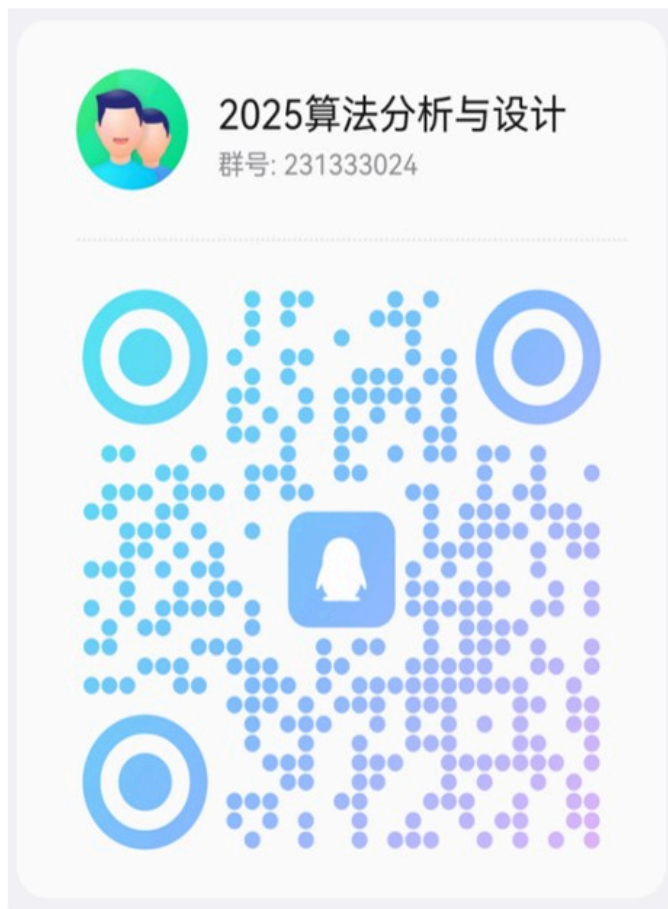
Office: 一综B323

华中农业大学信息学院

# 准备工作



- 请未入群同学加入2025算法分析与设计课程群



群名：2025算法分析与设计

群号：231333024

# 教学目的、内容和形式



- 目的：
  - 掌握算法的特性；
  - 掌握分析算法的基本方法；
  - 通过学习常用的精巧算法, 掌握设计算法的基本方法.
- 教材
  - 《算法设计技巧与分析》，M.H. Alsuwaiyel，电子工业出版社.
  - 《算法设计与分析——C++语言描述》，陈慧南，电子工业出版社
- 参考书：
  - 《算法设计与分析》，王晓东编著，清华大学出版社.
  - 《计算机算法基础》，余详宣等编著，华中科技大学出版社
- 学习形式：
  - 讲授、实验相结合.

# 成绩评定办法



- 平时作业 (30%)
- 综合大作业 (30%)
- 期末考试 (40%)

# 本书主要内容



## Part 1 算法的基本概念

1. 算法分析概念
2. 数学基础
3. 数据结构
4. 堆与不相交集

## Part 2 基于递归的技术

1. 归纳
2. 分治
3. 动态规划

## Part 3 最先割技术

- 贪心算法

## Part 4 问题复杂性

NP完全问题

## Part 5 克服困难性

回溯法

# 第一章

## 算法分析的基本概念





# 本章的主要内容

1. 引言
2. 搜索问题  
二分搜索
3. 排序问题  
合并两个有序表  
选择排序  
从底向上合并排序
4. 算法分析概念  
时间复杂度, 空间复杂度  
最优算法  
如何估算算法的运行时间  
最坏和平均情况分析



## 什么是算法?



- 算法是定义好的能够完成一定任务的有限指令集合，通常它由一个初始化状态，终止于一个对应的可识别的结束状态。
- 算法通常由计算机程序来实现，算法设计的错误可能导致程序运行时的失败。





# 算法的5个特征

## 1. 有效性

算法必须在有限的时间能够完成，甚至用纸和笔完成

## 2. 确定性

算法的每一步能够清楚的定义

## 3. 有限性

算法能够在有限的步骤完成

## 4. Input

算法有0个或者多个输入

## 5. Output

算法有一个或者多个输出

- 满足有效性，确定性，输入，输出特征的程序是一个过程而不是算法，举例：操作系统是过程而不是算法

# 算法的比较



对于同一个问题，可能存在多个算法。那么哪个算法好一点呢？

- 解决问题所需要的时间和空间
- 其它一些所需要的资源，例如通讯开销，处理器的数目，等等



## 1.3 二分搜索

### 搜索问题描述

设 $A[1..n]$  为一个包含 $n$ 个元素的数组. 考虑这样的问题: 判断给定元素 $x$ 是否在 $A$ 中。结果就是找到这个元素的下标 $j$ ,  $1 \leq j \leq n$ , 使得 $x = A[j]$  ; 如果不存在这个元素,  $j=0$

# 线性搜索



## 主要思想

- 扫描数组A，依次将每个元素和x进行比较。
- 如果经过j次比较， $1 \leq j \leq n$ ，找到x， $x=A[j]$ ，则返回下标 j
- 否则返回下标0，表示搜索失败了



## 例子

- 考虑下面的数组

$A[1..14] = 1, 4, 5, 7, 8, 9, 10, 12, 15, 22, 23, 27, 32, 35$

我们需要搜索  $x=22$ .

| 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
|---|---|---|---|---|---|----|----|----|----|----|----|----|----|
| 1 | 4 | 5 | 7 | 8 | 9 | 10 | 12 | 15 | 22 | 23 | 27 | 32 | 35 |



$A[10]=22!$

?

如果数组A中的元素是有序的，是否存在更为有效的算法





## Algorithm 1.1 LINEARSEARCH

**Input:** An array  $A[1 \dots n]$  of  $n$  elements and an element  $x$ .

**Output:**  $j$  if  $x = A[j]$ ,  $1 \leq j \leq n$ , and 0 otherwise.

1.  $j \leftarrow 1$
2. **while**  $(j < n)$  **and**  $(x \neq A[j])$
3.  $j \leftarrow j + 1$
4. **end while**
5. **if**  $x = A[j]$  **then return**  $j$  **else return** 0.

如果数组A中的元素不是有序的，则可能需要扫描所有元素

如果是有序的，是否存在更为有效的算法？？？





# 二分搜索

## 基本思想

- 设  $A[low...high]$  为一个非递减元素排序的数组。其中  $A[mid]$  表示  $A$  的中间位置的元素；假设  $x > A[mid]$ 。
- 如果在数组  $A$  中, 那么  $x$  在  $A[mid]$  的后面
- 我们进一步在子集  $A[mid+1...high]$  中搜索  $x$

## 例子

- 我们在下面数组中搜索  $x=22$

$A[1..14] =$

1,4,5,7,8,9,10,12,15,22,23,27,32,35.

| 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | low | high | mid |
|---|---|---|---|---|---|----|----|----|----|----|----|----|----|-----|------|-----|
| 1 | 4 | 5 | 7 | 8 | 9 | 10 | 12 | 15 | 22 | 23 | 27 | 32 | 35 | 1   | 14   | 7   |
|   |   |   |   |   |   |    |    |    |    |    |    |    |    | 8   | 14   | 11  |
|   |   |   |   |   |   |    |    |    |    |    |    |    |    | 8   | 10   | 9   |
|   |   |   |   |   |   |    |    |    |    |    |    |    |    | 10  | 10   | 10  |

success!

# 例子

在下面数组中搜索  $x=21$

$A[1..14] = 1, 4, 5, 7, 8, 9, 10, 12, 15, 22, 23, 27, 32, 35.$

| 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | low | high | mid |
|---|---|---|---|---|---|----|----|----|----|----|----|----|----|-----|------|-----|
| 1 | 4 | 5 | 7 | 8 | 9 | 10 | 12 | 15 | 22 | 23 | 27 | 32 | 35 | 1   | 14   | 7   |
|   |   |   |   |   |   |    |    |    |    |    |    |    |    | 8   | 14   | 11  |
|   |   |   |   |   |   |    |    |    |    |    |    |    |    | 8   | 10   | 9   |
|   |   |   |   |   |   |    |    |    |    |    |    |    |    | 10  | 10   | 10  |
|   |   |   |   |   |   |    |    |    |    |    |    |    |    | 10  | 9    |     |

failure!



## Algorithm 1.2 BINARYSEARCH

**Input:** 数组  $A[1 \dots n]$  包含  $n$  个非递减次序的元, 查找元素  $x$ .

**Output:**  $j$  if  $x = A[j]$ ,  $1 \leq j \leq n$ , and 0 otherwise.

1.  $low \leftarrow 1$ ;  $high \leftarrow n$ ;  $j \leftarrow 0$
2. **while** ( $low \leq high$ ) **and** ( $j = 0$ )
3.    $mid \leftarrow \lfloor low + high / 2 \rfloor$
4.   **if**  $x = A[mid]$  **then**  $j \leftarrow mid$
5.   **else if**  $x < A[mid]$  **then**  $high \leftarrow mid - 1$
6.   **else**  $low \leftarrow mid + 1$
7. **end while**
8. **return**  $j$

# 二分搜索算法分析



 定理：规模为 $n$ 的二分搜索算法中元素比较次数最多  $\lfloor \log n \rfloor + 1$ .

什么时候达到最大的比较次数呢？

- 在 `while` 循环的第  $j$  次循环时，剩余元素的数目是  $\lfloor n/2^{j-1} \rfloor$ 。
- 循环最大次数是满足条件  $\lfloor n/2^{j-1} \rfloor = 1$  时的  $j$  值，因此， $1 \leq n/2^{j-1} < 2 \rightarrow j = \lfloor \log n \rfloor + 1$





## 1.5 选择排序

📚 基本思想: 设  $A[1..n]$  为包含  $n$  个元素的数组

- 首先, 我们找到数组中的最小元素, 将它存在  $A[1]$ .
- 接着, 我们从剩下的  $n-1$  个元素中找到最小的元素, 将它存在  $A[2]$ .
- 重复以上过程, 知道整个数组中第二大的元素存在  $A[n-1]$ , 则算法停止.





## An example

- $A[1..11] = 6, 10, 9, 5, 3, 11, 4, 8, 1, 2, 7$

|   |    |   |   |   |    |   |   |   |   |   |
|---|----|---|---|---|----|---|---|---|---|---|
| 6 | 10 | 9 | 5 | 3 | 11 | 4 | 8 | 1 | 2 | 7 |
|---|----|---|---|---|----|---|---|---|---|---|

**i**

**j**

**k**

.....



## Algorithm 1.4 SELECTIONSORT

**Input:** 包含 $n$ 个元素的数组  $A[1..n]$ .

**Output:**  $A[1..n]$  中的元素非递减排列

1. **for**  $i \leftarrow 1$  **to**  $n-1$
2.      $k \leftarrow i$
3.     **for**  $j \leftarrow i+1$  **to**  $n$    {找到第  $i$  个最小元素}
4.         **if**  $A[j] < A[k]$  **then**  $k \leftarrow j$
5.     **end for**
6.     **if**  $k \neq i$  **then** 交换  $A[i]$  and  $A[k]$
7. **end for**



**Observation 1.3:** 选择排序算法的元素比较次数为 $n(n-1)/2$ . 元素赋值的次数介于 0 和  $3(n-1)$  之间.



## 1.6 插入排序

 基本思想:

- 从只包含一个元素的子数组 $A[1]$ 开始
- 接着, 将 元素 $A[2]$ 插入到子数组 $A[1]$ 的合适的位置, 这个位置可能在 $A[1]$ 之前或者 $A[1]$ 之后, 那么 $A[1..2]$  变为有序的子数组
- 然后, 插入 $A[3]$ ,  $A[4]$ , 直到所有的元素都插入, 整个数组变为有序.



## An example

- $A[1..11] = 6, 10, 9, 5, 3, 11, 4, 8, 1, 2, 7$

x =

10

|   |    |   |   |   |    |   |   |   |   |   |
|---|----|---|---|---|----|---|---|---|---|---|
| 6 | 10 | 9 | 5 | 3 | 11 | 4 | 8 | 1 | 2 | 7 |
|---|----|---|---|---|----|---|---|---|---|---|

i

j

...





## Algorithm 1.5 INSERTIONSORT

**Input:** An array  $A [1..n]$  of  $n$  elements.

**Output:**  $A [1..n]$  sorted in nondecreasing order.

```
1. for  $i \leftarrow 2$  to  $n$ 
2.    $x \leftarrow A[i]$ 
3.    $j \leftarrow i-1$ 
4.   while  $(j > 0)$  and  $(A[j] > x)$ 
5.      $A[j+1] \leftarrow A[j]$ 
6.      $j \leftarrow j-1$ 
7.   end while
8.    $A[j+1] = x$ 
9. end for
```

# 算法分析



 **Observation 1.4:** 插入排序的元素比较次数介于  $n-1$  和  $n(n-1)/2$  之间. 元素赋值次数等于元素比较次数加上  $n-1$ .



## 1.4 合并两个有序的数组

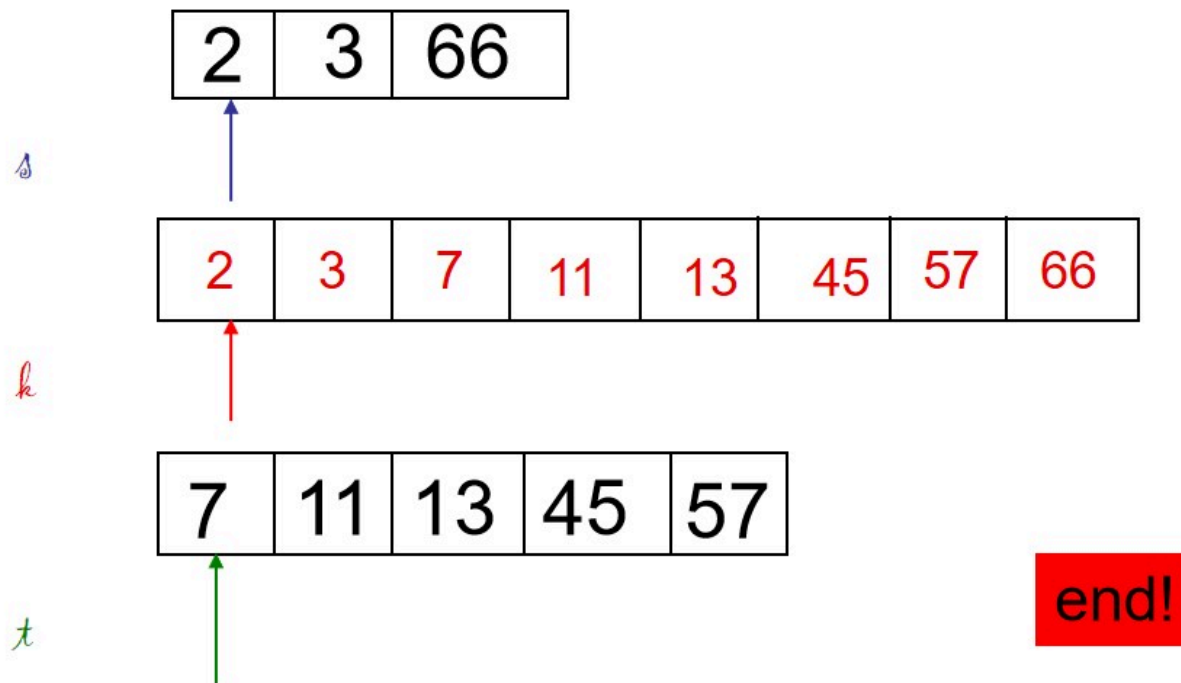
### ? 问题描述

假设我们有一个数组  $A[1 \cdots m]$  和三个下标  $p$ ,  $q$  和  $r$ , 其中  $1 \leq p \leq q < r \leq m$ . 该数组的两个子部分  $A[p \cdots q]$  和  $A[q+1 \cdots r]$  都是非递减排序. 我们需要重新调整  $A$  中的元素, 使得  $A[p \cdots r]$  中的元素非递减排序.



## 例子

- 合并两个子数组 2, 3, 66 and 7, 11, 13, 45, 57.





## Algorithm 1.3 MERGE

**Input:** 数组  $A[1...m]$  和三个下标  $p, q$  和  $r$ , 其中  $1 \leq p \leq q < r \leq m$ . 该数组的两个子部分  $A[p...q]$  和  $A[q+1...r]$  都是非递减排序

**Output:**  $A[p...r]$  中的元素非递减排序.

1. **comment:**  $B[p...r]$  是一个辅助数组

2.  $s \leftarrow p; t \leftarrow q + 1; k \leftarrow p$

3. **while**  $s \leq q$  **and**  $t \leq r$

4.     **if**  $A[s] \leq A[t]$  **then**

5.          $B[k] \leftarrow A[s]; s \leftarrow s+1$

6.     **else**  $B[k] \leftarrow A[t]; t \leftarrow t+1$

7.     **end if**

8.      $k \leftarrow k+1$

9. **end while**

10. **if**  $s = q + 1$  **then**  $B[k...r] \leftarrow A[t...r]$

11. **else**  $B[k...r] \leftarrow A[s...q]$

12. **end if**

13.  $A[p...r] \leftarrow B[p...r]$





Observation 1.1 : 对于合并长度为  $n_1$  和  $n_2$  的两个数组 ( $n_1 \leq n_2$ ,  $n = n_1 + n_2$ ), merge 算法的元素比较次数介于  $n_1$  和  $n-1$  之间. 特别的是, 如果两个数组的大小为  $\lfloor n/2 \rfloor$ , 元素比较次数介于  $\lfloor n/2 \rfloor$  和  $n-1$  之间.



Observation 1.2: 元素赋值次数为  $2n$ .





## 1.7 从底向上合并排序



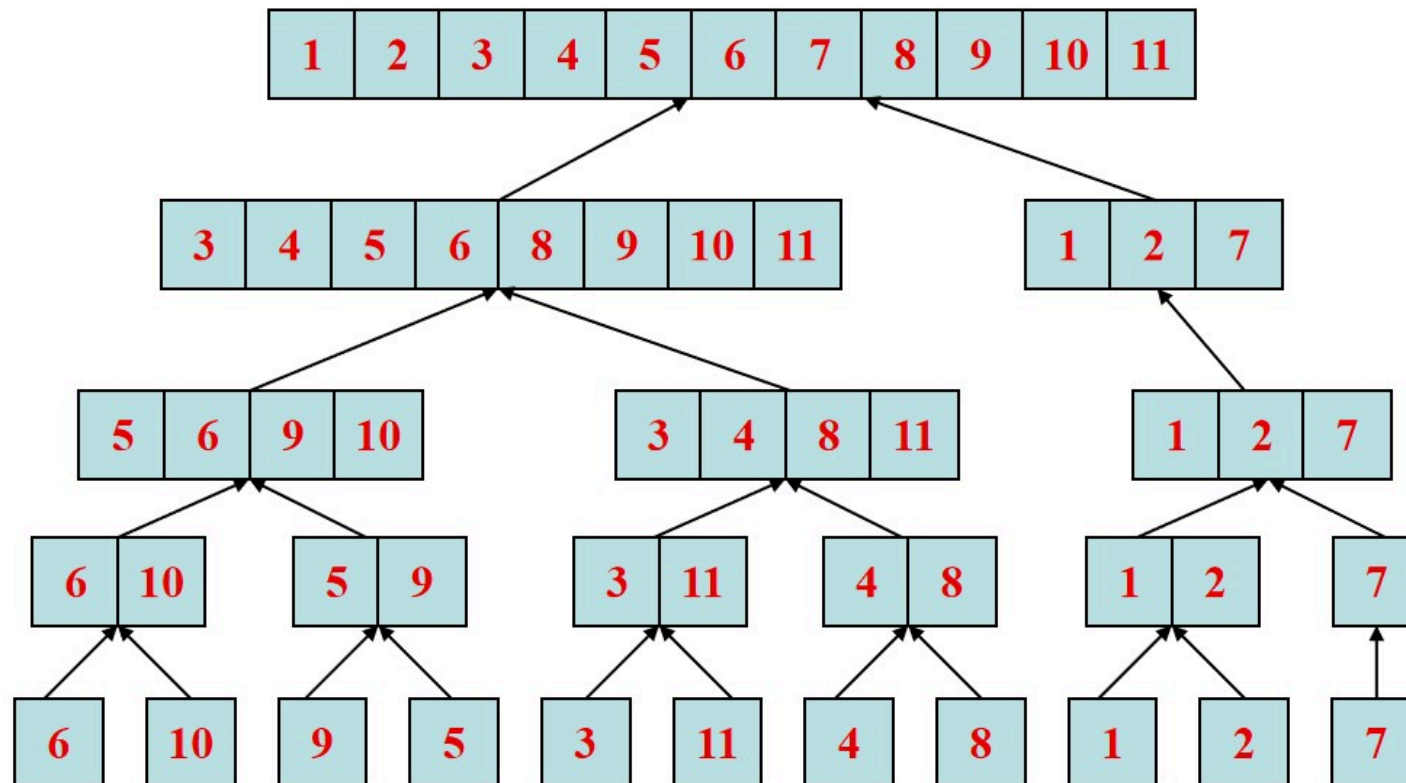
### 基本思想

- 直接在数组上进行操作,
- 利用两个有序数组合并的算法
- 初始化变量,  $s=1$ , 每一次循环  $s=2*s$
- $i+1$ ,  $i + s$  和  $i+t$  定义两个需要合并的子序列的边界.

# An example

success!

- $A[1..11] = 6, 10, 9, 5, 3, 11, 4, 8, 1, 2, 7$





## Algorithm 1.6 BOTTOMUPSORT

**Input:** 包含 $n$ 个元素的数组  $A[1..n]$

**Output:**  $A[1..n]$  按照非递减排序.

1.  $t \leftarrow 1$

2. **while**  $t < n$

3.    $s \leftarrow t$ ;  $t \leftarrow 2s$ ;  $i \leftarrow 0$

4.   **while**  $i + t \leq n$

5.     MERGE( $A, i+1, i + s, i+t$ )

6.      $i \leftarrow i+t$

7.   **end while**

8.   **if**  $i + s < n$  **then** MERGE( $A, i+1, i + s, n$ )

9. **end while**



# 算法分析

当 $n$ 是2的幂, 在这种情况下外层循环次数  $k = \log n$ , 在第 $j$ 次循环时, 对于两个规模为 $2^{j-1}$  子序列, 合并操作进行次数为 $n/2^j$

• **Observation 1.5:** 从底向上合并排序对规模为 $n$ 的数组进行操作时( $n$ 是2的幂), 元素比较次数介于 $(n \log n)/2$ 和 $n \log n - n + 1$ 之间. 元素赋值次数为 $2n \log n$ .



## 1.8 时间复杂度

- 时间复杂度不是程序运行所花费的实际时间，而是对于输入规模  $n$  的算法元运算的次数. Example:  $f(n) = n^2 \log n + 10n^2 + n$
- Definition 1.1: 对于任何计算步骤, 它的代价有一个时间常量作为上界, 而不管输入数据或执行的算法, 我们称之为“元运算”
- Examples:
  - 算法运算符:  $+$ ,  $-$ ,  $*$ ,  $/$
  - 元素比较和逻辑元算
  - 元素赋值



# 渐近符号



- $O(g(n))$ : 增长次数小于等于 $g(n)$  以及其常数倍的函数集合
  - $n \in O(n^2)$ ,  $100n+5 \in O(n^2)$ ,  $n(n-1)/2 \in O(n^2)$
- $\Omega(g(n))$ : 增长次数大于等于 $g(n)$  以及其常数倍的函数集合
  - $n^3 \in \Omega(n^2)$ ,  $0.00001n^3 \in \Omega(n^2)$ ,  $n^4+n+1 \in \Omega(n^2)$
- $\Theta(g(n))$ : 增长次数等于 $g(n)$  以及其常数倍的函数集合
  - $an^2+bn+c$  在 $a>0$ 的情况下都包含在 $\Theta(n^2)$  中
  - $n^2+\sin a$ ,  $n^2+\log n$  ?





# 0符号

- 定义1.2: 把函数  $f(n)$  包含在  $O(g(n))$ , 记作  $f(n) \in O(g(n))$ 。它成立的条件是: 对于所有足够大的  $n$ ,  $f(n)$  的上界由  $g(n)$  的常数倍所确定
  - 即, 存在常数  $c > 0$  和一个自然数  $n_0$ , 使得
$$\forall n \geq n_0, f(n) \leq c g(n).$$
- 例: 证明  $100n+5 \in O(n^2)$ 
  - $100n+5 \leq 100n+n$  (当  $n \geq 5$ )  $= 101n \leq 101n^2$   
取  $c=101, n_0=5$
  - $100n+5 \leq 100n+5n$  (当  $n \geq 1$ )  $= 105n \leq 105n^2$   
取  $c=105, n_0=1$



# $\Omega$ 符号

- 定义1.2: 把函数 $f(n)$ 包含在 $\Omega(g(n))$ , 记作 $f(n) \in \Omega(g(n))$ 。它成立的条件是: 对于所有足够大的 $n$ ,  $f(n)$ 的下界由 $g(n)$ 的常数倍所确定

- 即, 存在常数 $c > 0$ 和一个自然数 $n_0$ , 使得

$$\forall n \geq n_0, f(n) \geq c g(n).$$

- 例: 证明 $n^3 \in \Omega(n^2)$

当 $n \geq 0$ 时,  $n^3 \geq n^2$

取 $c=1$ ,  $n_0=1$



## Θ 符号

- 定义1. 3: 把函数 $f(n)$ 包含在 $\Theta(g(n))$ , 记作 $f(n) \in \Theta(g(n))$ 。它成立的条件是: 对于所有足够大的 $n$ ,  $f(n)$ 的下界, 上界由 $g(n)$ 的常数倍所确定

- 即, 存在常数 $c > 0$ 和一个自然数 $n_0$ , 使得

$$\forall n \geq n_0, c_1 g(n) \leq f(n) \leq c_2 g(n).$$

- 例: 证明 $n(n-1)/2 \in \Theta(n^2)$ 
  - 当 $n \geq 0$ 时,  $n(n-1)/2 = n^2/2 - n/2 \leq n^2/2$  —— 证明上界
  - $n(n-1)/2 = n^2/2 - n/2 \geq n^2/2 - n^2/2$  (当 $n \geq 2$ )  $= n^2/4$  —— 下界  
取 $c_2 = 1/2$ ,  $c_1 = 1/4$ 及 $n_0 = 2$



# 利用极限比较增长率

- 比较两个特定函数的增长率

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & \text{表明} t(n) \text{的增长率比} g(n) \text{小} & \text{①} \\ c > 0 & \text{表明} t(n) \text{的增长率和} g(n) \text{相同} & \text{②} \\ \infty & \text{表明} t(n) \text{的增长率比} g(n) \text{大} & \text{③} \end{cases}$$

- 情况①②意味着  $t(n) \in O(g(n))$
- 情况②③意味着  $t(n) \in \Omega(g(n))$
- 情况②意味着  $t(n) \in \Theta(g(n))$
- 利用基于极限的方法比基于定义的方法更方便

- 洛必达法则

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{t'(n)}{g'(n)}$$

- 史特林公式

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$





# 利用极限比较增长率

- 例：比较 $\log_2 n$ 和 $\sqrt{n}$ 的增长次数

$$\lim_{n \rightarrow \infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{(\log_2 n)'}{(\sqrt{n})'} = \lim_{n \rightarrow \infty} \frac{(\log_2 e) \frac{1}{n}}{\frac{1}{2\sqrt{n}}} = 2 \log_2 e \lim_{n \rightarrow \infty} \frac{\sqrt{n}}{n} = 0$$

- 例：比较 $n!$ 和 $2^n$ 的增长率

$$\lim_{n \rightarrow \infty} \frac{n!}{2^n} = \lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n}{2^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \frac{n^n}{2^n e^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \left(\frac{n}{2e}\right)^n = \infty$$





## 1.9 空间复杂度

- 为了求解问题的实例而执行的计算步骤所需要的内存空间,它不包括分配用来存储输入的空间. 换句话说,就是算法需要的工作空间
- 不包括输入大小的原因是为了区分那些在整个计算过程中占用了'少于'线形空间的算法.



# 例子

• 在线性搜索算法中, 只需要一个内存单元用来存储搜索结果. 我们可以说需要的空间为 $\Theta(1)$ . 二分搜索, 选择排序和插入排序的情况一样.

• 更多的例子 (see P20)



## 1.10 最优算法

 定义:一般说来,如果我们能够这个证明解决问题 $\Pi$ 的时间复杂度为 $\Omega(f(n))$ ,那么一个时间复杂度为 $O(f(n))$ 的算法称为一个最优算法  
(这个定义没有考虑空间复杂度)



## 1.11 怎么估算算法的运行时间

- 计算迭代的次数.
- 计算基本操作的频度.
- 使用递推关系



## 1.11.1 计算迭代的次数

例子:

**Input:**  $n=2^k$ , for some positive integer  $k$ .

**Output:** count=number of times Step 4 is executed.

```
1. count  $\leftarrow$  0
2. while  $n \geq 1$ 
3.     for  $j \leftarrow 1$  to  $n$ 
4.         count  $\leftarrow$  count + 1
5.     end for
6.      $n \leftarrow n/2$ 
7. end while
8. return count
```





## 分析

- COUNT1包含两个嵌套循环和一个变量count, 这个变量用来对算法执行的迭代次数计数, 这里  $n=2^k$ , 其中  $k$  为正整数.

while循环执行  $k+1$  次, 这里  $k=\log n$ . For循环  
执行  $n$  次, 之后是  $n/2, n/4, \dots, 1$ .

$$\sum_{j=0}^k \frac{n}{2^j} = n \sum_{j=0}^k \frac{1}{2^j} = n(2 - \frac{1}{2^k}) = 2n - 1 = \Theta(n)$$



## 1.11.2 计算基本运算的频度

- **Definition 1.6:** 如果算法中的一个元运算具有最高频率, 所以其他元运算频率均在他的频率的常数倍内, 则在这个元运算称为基本运算
- 元运算如何确定?
  - ✓ 在搜索和排序算法中, 元运算一般是元素比较
  - ✓ 在矩阵乘法运算中, 数量乘法运算是元运算
  - ✓ 在遍历一个链表时, 元运算是设置或更新指针
  - ✓ 在图遍历中, 元运算是访问节点的“动作”和被访问节点的计数



## 1.11.2 计算基本运算的频度



- Example: MERGE算法中的元素赋值是基本运算. 注意元素比较一般意义下不是基本运算, 极端情况下可能只有一次





## 1.11.2 计算基本运算的频度

- 确定算法BOTTOMUPSORT的确界
- 首先，这种算法的基本运算从MERGE算法继承而来，在while循环中每次循环都调用算法MERGE，所以，我们可以选择元素比较作为元运算
- 当 $n$ 是2的幂时，BOTTOMUPSORT的元素比较次数在  $(n \log n) / 2$  和  $n \log n - n + 1$  之间。这意味着，元素比较次数是  $\Theta(n \log n)$
- 由于算法用到的元素比较运算，在相差一个常数因子的意义下具有最大频度，我们可以得出结论，算法运行时间和比较次数成正比。所以算法运行时间是  $\Theta(n \log n)$



## 1.11.3 使用递推关系

**Input:** An array  $A[1..n]$  of  $n$  elements sorted in nondecreasing order and an element  $x$ .

**Output:**  $j$  if  $x=A[j]$ ,  $1 \leq j \leq n$ , and 0 otherwise.

1. *binarysearch*(1, $n$ )

**Procedure** *binarysearch*(low,high)

1. **if**  $low > high$  **then return** 0

2. **else**

3.  $mid \leftarrow \lfloor (low+high)/2 \rfloor$

4. **if**  $x=A[mid]$  **then return**  $mid$

5. **else if**  $x < A[mid]$  **then return** *binarysearch*(low, $mid-1$ )

6. **else return** *binarysearch*( $mid+1$ ,high)

7. **end if**





# BINARYSEARCH的递归算法

- 令 $C(n)$ 为一个大小是 $n$ 的实例中执行比较的次数，则算法所做比较次数的递推公式为：

$$C(n) \leq \begin{cases} 1 & \text{if } n = 1 \\ C(\lfloor n/2 \rfloor) + 1 & \text{if } n \geq 2 \end{cases}$$

$$\begin{aligned} C(n) &\leq C(\lfloor n/2 \rfloor) + 1 \\ &= C(\lfloor \lfloor n/2 \rfloor / 2 \rfloor) + 1 + 1 \\ &= C(\lfloor n/4 \rfloor) + 1 + 1 \\ &\dots\dots \\ &= \lfloor \log n \rfloor + 1 \end{aligned}$$

$$C(n) = O(\log n)$$



## 1.12 最坏情况和平均情况分析

- 很多时间, 我们很难用一个函数来表示算法对于输入规模 $n$ 的时间复杂度
- 因此, 我们转而使用
  - 最坏情况分析
  - 平均情况分析
  - 最好情况分析?



## 1.12.1 最坏情况分析

- 考虑对于输入规模为n的问题的最大代价

- 例子



- 以插入排序为例, 当给定的原序列中数据递降排列时, 所需要的比较次数最多
- 最坏情况的时间复杂度  $\Theta(n^2)$



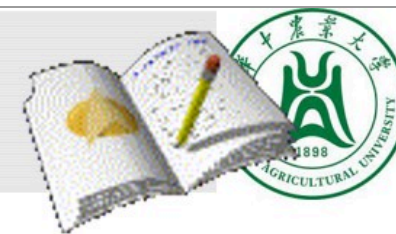
## 1.12.2 平均情况分析

- LINEARSEARCH算法中元素的比较次数在 $1 \sim n$ 之间, 有 $n$ 中可能性, 每种可能性发生的概率为  $1/n$
- 在平均情况下, 元素比较次数为

$$T(n) = \sum_{j=1}^n j \times \frac{1}{n} = \frac{1}{n} \sum_{j=1}^n j = \frac{1}{n} \frac{n(n+1)}{2} = \frac{n+1}{2}$$

故平均情况下的时间复杂度为 $\Theta(n)$ 。

## 本章小结



- 掌握算法的基本概念和具有的特征
- 掌握渐进符号的含义
- 重点掌握时间复杂度的概念，能够使用渐进符号表示
- 能够分析算法的时间复杂度和空间复杂度





# Exercises

- 1.13
- 1.16
- 1.38